# 51. AWS (Amazon Web Services): Full Theory in Simple Words

## Contents


1. What is the Cloud?
2. What is AWS?
3. Why AWS was introduced (history)
4. Why companies use AWS
5. Service models: IaaS, PaaS, SaaS, Serverless
6. Regions, Availability Zones, Edge Locations
7. The most important AWS services (map)
8. Compute: EC2, Lambda, ECS, Beanstalk
9. Storage: S3, EBS
10. Databases: RDS, DynamoDB, ElastiCache
11. Networking: VPC, Security Groups, ELB, Route 53, CloudFront
12. Security: IAM
13. Monitoring and messaging: CloudWatch, SQS, SNS
14. Shared Responsibility Model
15. Pricing, Free Tier and avoiding surprise bills
16. Ways to talk to AWS: Console, CLI, SDK
17. Deploy a NestJS app: 4 ways
18. Full example: NestJS + RDS Postgres + Redis + S3
19. Typical production architecture
20. AWS vs Azure vs GCP
21. Common mistakes
22. Quick summary

# 1. What is the Cloud?

**The cloud means using someone else's computers over the internet, and paying only for what you use.**

> **Real-life example: electricity.** You don't build a power plant at home. You plug into the wall and pay a monthly bill for the units you used.
> The cloud is the same: you don't buy servers. You "plug in" over the internet and pay for the hours or GBs you used.

| Own servers (on-premise) | Cloud |
|---|---|
| Buy hardware upfront (lakhs of ₹) | No upfront cost; pay per hour or second |
| Takes weeks to buy and set up | Ready in minutes |
| You fix broken disks, power, cooling | The provider handles the hardware |
| Traffic spike? Buy more servers (slow) | Traffic spike? Add servers in seconds (auto scaling) |
| Servers sit idle at night but you still paid | Turn them off and stop paying |

# 2. What is AWS?

**AWS (Amazon Web Services) is Amazon's cloud platform.** It rents out computing power, storage, databases and 200+ other services over the internet.

It's the **biggest cloud provider in the world**. Netflix, Airbnb, Swiggy, Zomato, NASA and many startups run on AWS.

> In one line: **AWS = a huge shop where you rent servers, storage and databases by the hour instead of buying them.**

# 3. Why was AWS introduced? (history)

### The problem inside Amazon (early 2000s)

- Amazon.com was growing very fast. Every new team spent **months building the same basic infrastructure** (servers, databases, storage) before writing any real feature.
- Amazon had to buy enough servers for **peak traffic** (like holiday sales). For the rest of the year, most of those servers sat idle.
- Teams' systems were tangled together, so they moved slowly.

### The fix

Amazon rebuilt its infrastructure into **clean, reusable services with APIs** (storage as a service, compute as a service…). Then leaders including **Andy Jassy** (who later became Amazon CEO) realised that other companies have the same problem. They decided to **sell this infrastructure to the world**.

### Timeline

| Year | What happened |
|---|---|
| 2002 | First small "Amazon Web Services" launched for developers (product data APIs) |
| 2004 | **SQS** (message queue) launched, the first real infrastructure service |
| **2006** | **S3** (storage, March) and **EC2** (virtual servers, August) launched. This is **modern AWS's birthday**. |
| 2009 | **RDS**: managed databases |
| 2010 | Amazon.com itself moved fully onto AWS |
| 2012 | DynamoDB (NoSQL) |
| 2014 | **Lambda**: serverless (run code without managing servers). Also ECS for Docker containers. |
| 2018+ | EKS (Kubernetes), and later AI/ML services like SageMaker and Bedrock |

> **Why AWS, in one line:** Amazon turned the infrastructure it had built for itself into rentable services, so that any company can get servers in minutes, pay only for what it uses, and focus on its product instead of hardware.

## 4. Why companies use AWS

| Benefit | Simple meaning |
| --- | --- |
| **Pay-as-you-go** | No big upfront cost. You're billed per second, hour or GB. |
| **Scalability** | Go from 1 server to 100 when traffic grows, and back down |
| **Speed** | A new server or database is ready in minutes |
| **Global** | Run your app close to users in India, the US, Europe… |
| **Reliability** | Many data centres, so if one fails, others keep running |
| **Managed services** | AWS handles backups, patches and upgrades for you (e.g. RDS) |
| **Security** | Strong physical security, encryption and access control |

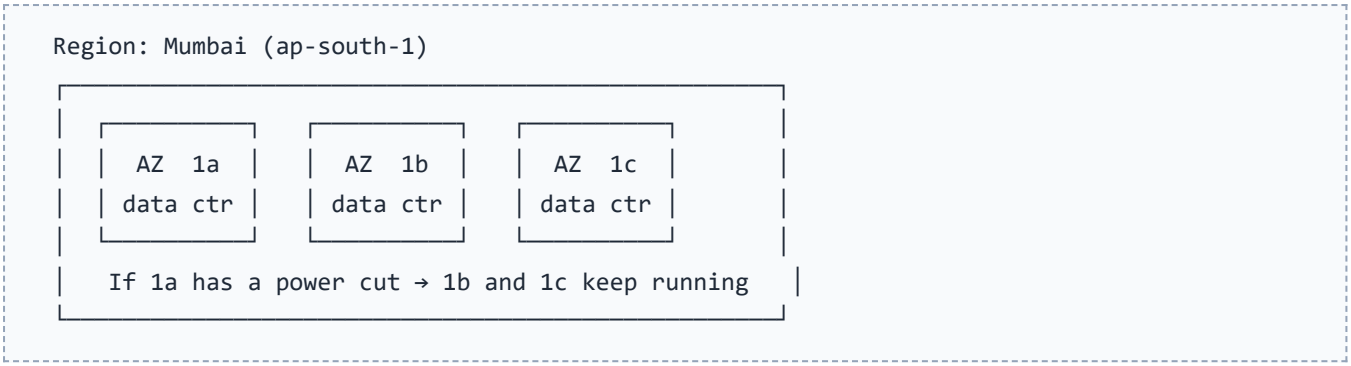
## 5. Service models

> **Pizza example** 🍕
> **On-premise:** make pizza at home (you do everything).
> **IaaS:** buy a ready base and bake it yourself (you get a machine and manage the rest).
> **PaaS:** pizza delivery (you just choose toppings, that is, your code).
> **SaaS:** eat at a restaurant (you just use the finished product).

| Model | You manage | AWS manages | AWS example |
|---|---|---|---|
| **IaaS** (Infrastructure) | OS, runtime, app, data | Hardware, network | EC2 |
| **PaaS** (Platform) | App code, data | OS, runtime, scaling | Elastic Beanstalk, RDS |
| **Serverless / FaaS** | Only functions | Everything else, and it scales to zero | Lambda |
| **SaaS** (Software) | Nothing, just use it | Everything | Gmail, Zoom (not AWS-specific) |

## 6. Regions, Availability Zones, Edge Locations

| Term | Meaning | Example |
|---|---|---|
| **Region** | A geographic area with AWS data centres | `ap-south-1` = Mumbai, `ap-south-2` = Hyderabad, `us-east-1` = N. Virginia |
| **Availability Zone (AZ)** | One or more separate data centres inside a region, with their own power and network | `ap-south-1a`, `ap-south-1b`, `ap-south-1c` |
| **Edge Location** | Small sites worldwide for caching content close to users (CDN) | Used by CloudFront |




**Choose a region** close to your users (Mumbai for India), allowed by any data laws you must follow, and with the services you need. Prices differ slightly by region.

## 7. Map of the most important services

| Category | Service | Simple meaning | Similar to |
|---|---|---|---|
| **Compute** | EC2 | Rent a virtual server | Your own laptop in the cloud |
| | Lambda | Run a function on demand, no server | Pay per call |
| | ECS / Fargate | Run Docker containers | Docker in the cloud |
| | Elastic Beanstalk | Upload code, and AWS sets up the servers | Heroku / Render |
| **Storage** | S3 | Store files (images, PDFs, backups) | Google Drive for apps |
| | EBS | Hard disk attached to EC2 | Laptop SSD |
| **Database** | RDS | Managed SQL (Postgres, MySQL) | Supabase / local Postgres |
| | DynamoDB | Managed NoSQL key-value | MongoDB-ish |
| | ElastiCache | Managed Redis | Your Docker Redis |
| **Network** | VPC | Your private network in AWS | Your home WiFi network |
| | Security Group | Firewall: which ports are open | Building security guard |
| | ELB / ALB | Load balancer: spreads traffic across servers | Traffic police |
| | Route 53 | DNS: domain name → server | Phone contacts list |
| | CloudFront | CDN: fast content delivery worldwide | Cloudflare |
| **Security** | IAM | Users, roles, permissions | Office ID cards |
| | Secrets Manager | Store passwords and API keys safely | A safe locker for `.env` |
| **Ops** | CloudWatch | Logs, metrics, alarms | CCTV + alarm |
| | SQS | Message queue | BullMQ / a token queue |
| | SNS | Notifications (email, SMS, push) | Broadcast/announcement |
| **Containers** | ECR | Store your Docker images | Private Docker Hub |

# 8. Compute

### EC2 (Elastic Compute Cloud): a virtual server

You rent a computer in AWS. You choose the OS (Ubuntu, Amazon Linux, Windows), the size, and the disk. You SSH into it and install anything, like Node, PM2 or Nginx.

| Term | Meaning |
|---|---|
| **Instance** | One running virtual server |
| **Instance type** | Size: `t3.micro` (2 vCPU, 1 GB) up to huge machines. "t" = cheap, general-purpose. |
| **AMI** | The OS image used to create the server (like a Docker image for a VM) |
| **Key pair** | A `.pem` file used to SSH in (keep it secret!) |
| **Elastic IP** | A fixed public IP (otherwise the IP changes when the instance stops) |
| **Auto Scaling Group** | Automatically adds or removes instances based on load |

Pricing options: **On-Demand** (pay per second, flexible), **Reserved / Savings Plans** (commit for 1–3 years, much cheaper), **Spot** (spare capacity, very cheap, but AWS can take it back).

### Lambda: serverless functions

Upload a function. It runs **only when called** (by an HTTP request, a file upload, a schedule...). There's no server to manage, it scales automatically, and you pay per request and run time.

```
// handler.js
export const handler = async (event) => {
  return { statusCode: 200, body: JSON.stringify({ msg: 'Hello from Lambda' }) };
};
```

> ✅ Great for small APIs, cron jobs and image processing. ❌ Each run has a time limit (max 15 minutes). It may have **cold starts** (a slow first call). It's not ideal for long-running or WebSocket-heavy apps.

### ECS + Fargate: run Docker containers

You push your Docker image to **ECR**. ECS runs it. With **Fargate**, you don't even manage the servers underneath; you just say "run 2 copies of this container with 0.5 CPU and 1 GB".

### Elastic Beanstalk: easiest for beginners

Upload your Node app (zip) or a Dockerfile. Beanstalk creates the EC2 instances, load balancer, auto scaling and monitoring for you.

### Which one to pick?

| You want... | Use |
|---|---|
| Full control, learn Linux | EC2 |
| Easiest "just deploy my app" | Elastic Beanstalk (or App Runner) |
| You already use Docker | ECS Fargate |
| Small functions, pay only when used | Lambda |
| Big microservices with Kubernetes | EKS |

# 9. Storage

### S3 (Simple Storage Service)

It stores **any file** (images, videos, PDFs, backups) with practically unlimited space. It's very durable (designed for 99.999999999%, "eleven 9s").

| Term | Meaning |
|---|---|
| **Bucket** | A top-level folder. Its name must be unique worldwide (e.g. `gourish-app-uploads`). |
| **Object** | A file plus its metadata |
| **Key** | The file's path/name, e.g. `students/12/photo.jpg` |
| **Storage classes** | Standard (frequent access), Infrequent Access, Glacier (archives, very cheap but slow to get back) |
| **Pre-signed URL** | A temporary link that lets a user upload or download one file without AWS keys |

> Buckets are **private by default**. Keep them private. Many data leaks happened because someone made a bucket public by mistake.

> You can also host a **static website** (an Angular/React build) on S3 + CloudFront.

### EBS (Elastic Block Store)

A virtual hard disk attached to an EC2 instance. If the server stops, the disk keeps its data. You can take **snapshots** (backups).

| S3 | EBS |
|---|---|
| Object storage, accessed over HTTP/API | Block disk, mounted on one EC2 |
| For files and uploads | For the OS, the app and DB files on that server |
| Unlimited, pay per GB used | Fixed size, pay for the size you chose |

# 10. Databases

| Service | Type | When to use |
| --- | --- | --- |
| **RDS** | SQL: PostgreSQL, MySQL, MariaDB, SQL Server, Oracle | Your TypeORM + Postgres apps |
| **Aurora** | AWS's own faster, Postgres/MySQL-compatible database | Bigger production apps |
| **DynamoDB** | NoSQL key-value, serverless | Huge scale, simple access patterns |
| **DocumentDB** | MongoDB-compatible | Mongoose-style apps |
| **ElastiCache** | Redis / Valkey / Memcached | Caching, sessions (see theory #49) |

> **Why RDS instead of installing Postgres on EC2?** RDS gives you **automatic backups, patching, Multi-AZ failover, read replicas and monitoring**. You just get a connection URL.

# 11. Networking

### VPC (Virtual Private Cloud)

Your own **private network** inside AWS. You decide which things are reachable from the internet.

| Part | Meaning |
|---|---|
| **Public subnet** | Reachable from the internet (load balancer, bastion host) |
| **Private subnet** | Not reachable from the internet (databases, Redis, app servers) |
| **Internet Gateway** | The door between the VPC and the internet |
| **NAT Gateway** | Lets private servers go *out* (e.g. `npm install`) without letting anyone in. ⚠ It costs money every hour. |

### Security Group: a firewall for each resource

Rules like "allow port 443 from anywhere" or "allow port 5432 **only** from the app servers' security group".

```
App server SG:  inbound 443/80 from Load Balancer SG
                inbound 22 (SSH) from MY IP only
RDS SG:         inbound 5432 from App server SG only   ✅ DB never public
```

### Load Balancer (ALB)

It spreads traffic across several servers and checks their health. If one dies, traffic goes to the others. It also handles **HTTPS** certificates (free via ACM, AWS Certificate Manager).

### Route 53 and CloudFront

**Route 53** = DNS: it points `api.myapp.com` to your load balancer. **CloudFront** = CDN: it caches images, JS and CSS at edge locations so pages load fast everywhere.

# 12. Security: IAM (Identity and Access Management)

IAM decides **who** can do **what** on **which** resource.

| Term | Meaning | Example |
| --- | --- | --- |
| **Root user** | The account owner's email login, with full power | The master key 🔑. Lock it away. |
| **User** | A person or app with long-term login or keys | `gourish-dev` |
| **Group** | A set of users sharing permissions | `developers` |
| **Role** | Temporary permissions that a service or person "wears" | EC2 role that can read S3 |
| **Policy** | A JSON document listing what's allowed or denied | see below |

```
{
  "Version": "2012-10-17",
  "Statement": [{
    "Effect": "Allow",
    "Action": ["s3:GetObject", "s3:PutObject"],
    "Resource": "arn:aws:s3:::gourish-app-uploads/*"
  }]
}
```

**IAM golden rules:**
1. Turn on **MFA** for root, and **never** use root for daily work.
2. Give **least privilege**: only the permissions that are needed.
3. Give AWS services **roles** rather than access keys (e.g. an EC2 role reads S3 with no keys in the code).
4. **Never** commit access keys to GitHub. Bots scan GitHub for keys within minutes and mine crypto on your bill.

## 13. Monitoring and messaging

| Service | What it does | Example |
|---|---|---|
| **CloudWatch Logs** | Collects your app's `console.log` output | See NestJS errors from ECS/Lambda |
| **CloudWatch Metrics/Alarms** | CPU, memory, request count, with alerts | "Email me if CPU > 80% for 5 min" |
| **SQS** | Queue: one service puts jobs in, another processes them | Order placed → queue → email worker |
| **SNS** | Pub/Sub: one message to many subscribers (email, SMS, Lambda) | Alert to the whole team |
| **EventBridge** | Event bus + cron schedules | Run a Lambda every night at 2 AM |

## 14. Shared Responsibility Model

| **AWS is responsible for... ("security of the cloud")** | **You are responsible for... ("security in the cloud")** |
|---|---|
| Physical data centres, guards, power | Your code and its bugs |
| Hardware, network cables, hypervisor | IAM users, passwords, MFA, keys |
| Managed service internals (e.g. the RDS engine patches) | Security group rules (open ports) |
| | OS updates on **EC2** |
| | Your data: encryption, backups, keeping buckets private |

> **Example:** AWS is the building owner (locks the main gate, CCTV). You are the tenant. If you leave **your flat's door** open, that's on you.

## 15. Pricing, Free Tier and avoiding surprise bills

You pay for **compute time** (per second or hour), **storage** (per GB-month), **requests**, and **data going out** to the internet. Incoming data is usually free.

**Free Tier:** New accounts get free usage or credits to learn with, such as small EC2 instances, some S3 storage, a small RDS instance, and a monthly free amount of Lambda requests. The exact offer has changed over time (newer accounts get a credit-based free plan), so always check **aws.amazon.com/free** for the current terms.

> **Protect yourself from surprise bills:**
> 1. Create an **AWS Budget** with an email alert (e.g. at $5) **on day 1**.
> 2. **Stop or terminate** EC2/RDS when you're not using them. Stopped RDS restarts by itself after 7 days!
> 3. Watch the hidden costs: **NAT Gateway**, **Elastic IP not attached**, **public IPv4 addresses**, **EBS snapshots**, **load balancers** left running.
> 4. Check **Billing** → **Cost Explorer** every week while learning.
> 5. Delete everything after a practice session. "Terminate" means really deleted.

There's a **pricing calculator** at calculator.aws to estimate costs before you build.

## 16. Ways to talk to AWS

| Way | Use |
|---|---|
| **Console** | The website (console.aws.amazon.com). Click around; good for learning. |
| **CLI** | The `aws` command in a terminal; good for scripts |
| **SDK** | Code libraries, e.g. `@aws-sdk/client-s3` for Node/NestJS |
| **Infrastructure as Code** | CloudFormation, AWS CDK, Terraform: describe servers in code files |

```
# Install AWS CLI v2, then:
aws configure                  # enter access key, secret, region (ap-south-1)
aws sts get-caller-identity    # who am I?
aws s3 ls                      # list buckets
aws s3 cp photo.jpg s3://gourish-app-uploads/
aws ec2 describe-instances
```

## 17. Deploy a NestJS app: 4 ways

### Way 1: EC2 (manual, best for learning)

```
# 1. Launch EC2: Ubuntu, t3.micro, key pair, SG: 22 (my IP), 80, 443
# 2. SSH in
ssh -i mykey.pem ubuntu@<public-ip>

# 3. Install Node + git + pm2
curl -fsSL https://deb.nodesource.com/setup_20.x | sudo -E bash -
sudo apt install -y nodejs git nginx
sudo npm i -g pm2

# 4. Get code and run
git clone https://github.com/gourishnaik/my-app.git
cd my-app && npm ci && npm run build
pm2 start dist/main.js --name api
pm2 save && pm2 startup        # restart on reboot

# 5. Nginx reverse proxy: port 80 → 3000
#    location / { proxy_pass http://localhost:3000; }
```

### Way 2: Docker on ECS Fargate (modern, recommended)

```
# 1. Create an ECR repository "my-app", then log in and push
aws ecr get-login-password --region ap-south-1 | docker login --username AWS \
  --password-stdin <account-id>.dkr.ecr.ap-south-1.amazonaws.com
docker build -t my-app .
docker tag my-app:latest <account-id>.dkr.ecr.ap-south-1.amazonaws.com/my-app:latest
docker push <account-id>.dkr.ecr.ap-south-1.amazonaws.com/my-app:latest

# 2. In ECS: create Cluster → Task Definition (image, port 3000, env vars,
#    CPU/RAM) → Service (Fargate, 2 tasks, attach ALB)
```

It uses the same Dockerfile from theory #50.

### Way 3: Elastic Beanstalk (easiest)

```
pip install awsebcli
eb init        # pick region + Node.js platform
eb create my-app-env
eb deploy      # every time you change code
```

### Way 4: Lambda (serverless)

Wrap Nest with `@codegenie/serverless-express` and deploy using the Serverless Framework or AWS SAM. It's cheap for low traffic, but watch out for cold starts.

# 18. Full example: NestJS + RDS + ElastiCache + S3

### .env (or better: Secrets Manager)

```
DB_HOST=mydb.xxxxxx.ap-south-1.rds.amazonaws.com
DB_PORT=5432
DB_USER=postgres
DB_PASS=********
DB_NAME=mydb
REDIS_URL=redis://my-redis.xxxxxx.cache.amazonaws.com:6379
AWS_REGION=ap-south-1
S3_BUCKET=gourish-app-uploads
```

### TypeORM with RDS

```
TypeOrmModule.forRoot({
  type: 'postgres',
  host: process.env.DB_HOST,
  port: +process.env.DB_PORT,
  username: process.env.DB_USER,
  password: process.env.DB_PASS,
  database: process.env.DB_NAME,
  autoLoadEntities: true,
  synchronize: false,             // ⚠ never true in production
  ssl: { rejectUnauthorized: false }, // RDS uses SSL (better: load the RDS CA bundle)
}),
```

### Upload a file to S3

```
npm install @aws-sdk/client-s3 @aws-sdk/s3-request-presigner
```

```
import { Injectable } from '@nestjs/common';
import { S3Client, PutObjectCommand, GetObjectCommand } from '@aws-sdk/client-s3';
import { getSignedUrl } from '@aws-sdk/s3-request-presigner';

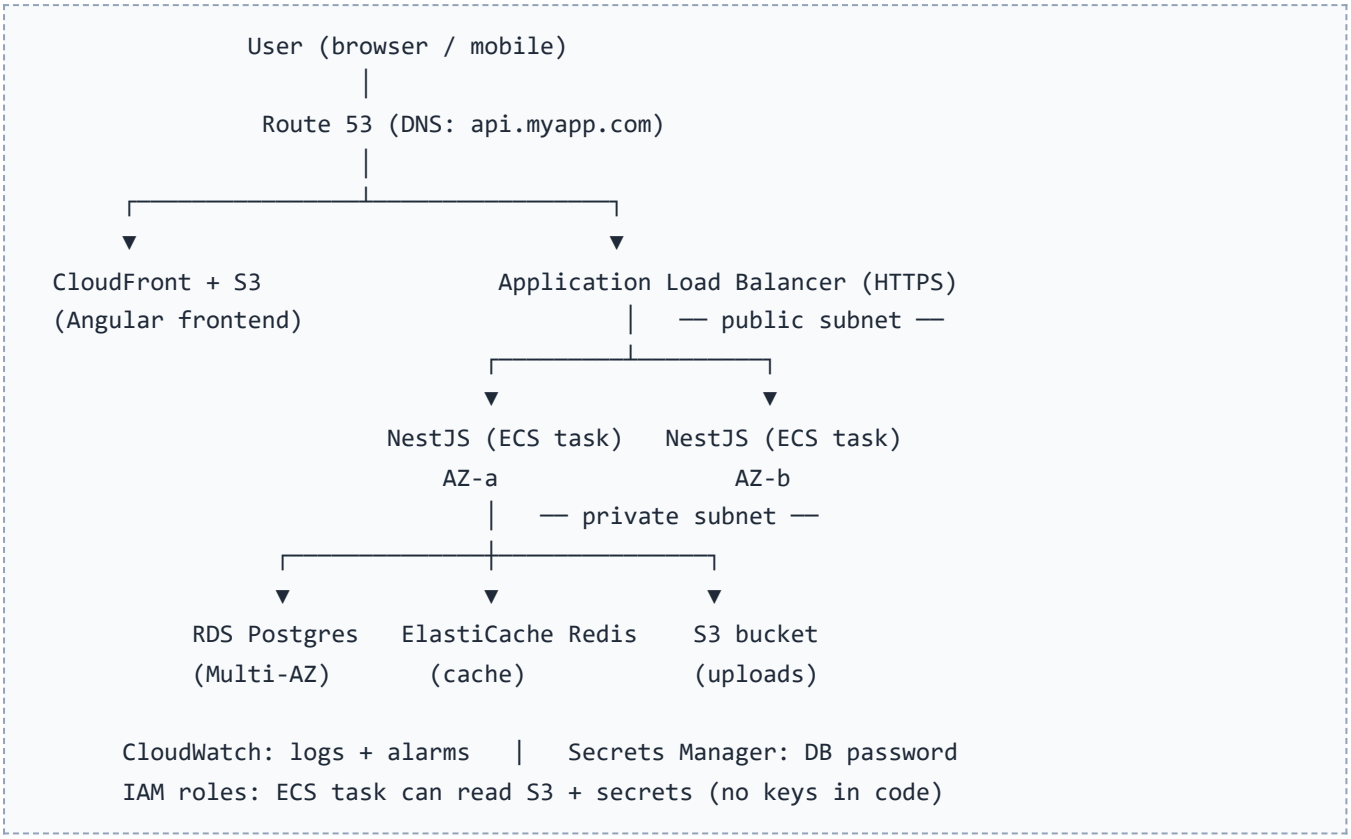
@Injectable()
export class FilesService {
  // No keys in code: on AWS it uses the IAM role,
  // locally it uses `aws configure` credentials.
  private s3 = new S3Client({ region: process.env.AWS_REGION });
  private bucket = process.env.S3_BUCKET;

  async upload(file: Express.Multer.File) {
    const key = `uploads/${Date.now()}-${file.originalname}`;
    await this.s3.send(new PutObjectCommand({
      Bucket: this.bucket,
      Key: key,
      Body: file.buffer,
      ContentType: file.mimetype,
    }));
    return { key };
  }

  // temporary download link, valid for 5 minutes
  async getDownloadUrl(key: string) {
    return getSignedUrl(
      this.s3,
      new GetObjectCommand({ Bucket: this.bucket, Key: key }),
      { expiresIn: 300 },
    );
  }
}
```

```
@Post('upload')
@UseInterceptors(FileInterceptor('file'))
upload(@UploadedFile() file: Express.Multer.File) {
  return this.filesService.upload(file);
}
```

## 19. Typical production architecture




| Piece | Why it's there |
|---|---|
| 2 AZs | If one data centre fails, the app stays up |
| Private subnets | The DB and Redis are never exposed to the internet |
| ALB | HTTPS, health checks, spreading load |
| CloudFront | Fast frontend worldwide |
| Secrets Manager + IAM roles | No passwords or keys in code or GitHub |

## 20. AWS vs Azure vs GCP

| Thing | AWS | Azure (Microsoft) | GCP (Google) |
|---|---|---|---|
| Virtual server | EC2 | Virtual Machines | Compute Engine |
| File storage | S3 | Blob Storage | Cloud Storage |
| Serverless | Lambda | Functions | Cloud Functions / Cloud Run |
| SQL DB | RDS | Azure SQL / Database for PostgreSQL | Cloud SQL |
| Kubernetes | EKS | AKS | GKE |
| Known for | Biggest, most services, first mover | Enterprise and Microsoft integration | Data, AI, Kubernetes (Google invented it) |

The concepts are the same everywhere. Learn one well and the others become easy.

## 21. Common beginner mistakes

| Mistake | Fix |
|---|---|
| Using the root account daily | Create an IAM user (or IAM Identity Center) with MFA |
| Access keys pushed to GitHub | Use roles and `.env` in `.gitignore`. If leaked, **deactivate the key immediately**. |
| No billing alert | Set up an AWS Budget on day 1 |
| Forgot to delete resources | Terminate EC2/RDS/NAT/ELB after practice |
| RDS made public with port 5432 open to `0.0.0.0/0` | Keep it in a private subnet, and allow it only from the app's SG |
| Wrong region selected in the console | Check the region dropdown (top right). Resources are per region! |
| `synchronize: true` in production | Use TypeORM migrations |
| App can't reach RDS (timeout) | Almost always a security group or subnet issue |

## 22. Quick summary

| Question | Answer |
|---|---|
| What is the cloud? | Renting computers over the internet; pay for what you use |
| What is AWS? | Amazon's cloud platform, the largest in the world, with 200+ services |
| When / why? | 2006 (S3 + EC2). Amazon turned its internal infrastructure into rentable services. |
| Region / AZ | Region = city/area (Mumbai). AZ = separate data centres inside it. |
| Server | EC2 |
| Files | S3 |
| Postgres | RDS |
| Redis | ElastiCache |
| Docker | ECR (store) + ECS Fargate (run) |
| Serverless | Lambda |
| Permissions | IAM: users, roles, policies, least privilege |
| Network/firewall | VPC + Security Groups |
| Logs | CloudWatch |
| Day-1 must do | MFA on root + billing budget alert |

**Learning path:** IAM → EC2 (deploy NestJS by hand) → S3 uploads → RDS Postgres → Docker + ECR + ECS → Load Balancer + domain + HTTPS → CloudWatch → (optional) AWS Certified Cloud Practitioner exam.